Exercise on Code Metrics and Technical Debt

Below are example solutions to the exercises. While these address the requirements, you are encouraged to explore alternative approaches and solutions. Your creativity and problem-solving strategies are valued :) (Solutions for the optional exercises are not provided)

Q2.1: Calculate the different internal quality metrics specified in the spreadsheet "`Internal Quality Exercise`" for each method in the source code.

Class	Method	Cyclomatic Complexity	Cognitive Complexity	Fan-in	Fan-out	Block depth
Account	Account	1	0	0	1	1
Account	deposit	2	1	0	3	2
Account	withdraw	2	1	0	3	2
Account	interestEarned	6	7	1	1	4
Account	sumTransactions	1	0	2	1	1
Account	checkIfTransactionsExist	2	1	1	0	2
Account	getAccountType	1	0	1	0	1
Bank	Bank	1	0	0	1	1
Bank	addCustomer	1	0	0	1	1
Bank	customerSummary	3	1	0	3	2
Bank	format	2	1	1	0	1
Bank	totalInterestPaid	2	1	0	1	2
Bank	getFirstCustomer	2	1	0	3	2
Customer	Customer	1	0	0	1	1
Customer	getName	1	0	2	0	1
Customer	openAccount	1	0	0	1	1
Customer	getNumberOfAccounts	1	0	1	1	1
Customer	totalInterestEarned	2	1	1	1	2
Customer	getStatement	2	1	0	3	2
Customer	statementForAccount	5	4	1	2	3
Customer	toDollars	1	0	3	2	1
DateProvider	getInstance	2	1	1	1	2
DateProvider	now	1	0	1	2	1
Transaction	Transaction	1	0	2	2	1

Q2.2: List two methods with the highest cyclomatic complexity. Justify what is causing the methods to have high complexity.

`interestEarned()`

- Multi-way branch: a switch(accountType) with 3 outcomes.
- Nested decisions: several if checks on amount inside cases (≤1000, ≤2000).

`statementForAccount(Account a)`

- Multi-way branch: switch(a.getAccountType()) with 3 cases.
- Loop: the for (Transaction t : …) introduces a decision each iteration.
- Inline conditional: ternary (t.amount < 0 ? … : …).

Q2.3: List two methods with the highest cognitive complexity. Justify what is causing the methods to have high complexity.

`interestEarned()`

- Each nested if increases cognitive load because the reader must keep track of multiple conditions and their scopes.

`statementForAccount(Account a)`

- The reader must mentally trace three levels of control flow (switch → loop → ternary).

Q2.4: Are the methods listed above the same for both types of complexity (cyclomatic and cognitive)? How similar/different are the actual complexity values for the methods?

Yes, the same two methods are the ones that look complex under both metrics. **But the numbers differ** because the metrics count different things.

`interestEarned()`

- Cyclomatic: 6 (multi-way switch counted by cases − 1, plus three ifs).
- Cognitive: 7 (the switch counts +1 total, but each nested if adds +1 plus +1 for nesting).

`statementForAccount(Account a)`

- Cyclomatic: 5 (switch + loop + ternary).
- Cognitive: 4 (switch +1, loop +1, ternary inside loop +2 for flow break + nesting).

Cyclomatic counts decision points; Cognitive emphasizes readability cost.

Q2.5: List two methods that can **cause more impact** if changed. Justify your choice based on internal quality metrics.

`toDollars` (fan-in=3); `sumTransactions`(fan-in=2).

Fan-in = number of other methods that call this method.
High fan-in → many dependents → a change can break or ripple through many callers.
Note: You can select any other method than `sumTransactions` with fan-in=2.

Q2.6: List two methods that are more likely to **be impacted** by a code change. Justify your choice based on internal quality metrics.
`withdraw` (fan-out=3); `deposit` (fan-out=3).
High fan-out → the method itself is complex or tightly coupled → is more likely to be impacted by a code change.
Note: You can select any other method than `deposit, withdraw` with fan-out=3.

Q2.7: Based on the answers above, explain which class you would choose to refactor before the next release? Justify your choice.
`Customer` class.

- It's a coupling hub: contains a high fan-out method `getStatement` (3) → highly sensitive to changes elsewhere, and a high fan-in utility toDollars (3) → many callers depend on it.
- `statementForAccount` also has non-trivial complexity (Cyclomatic 5, Cognitive 4) and deeper block depth (3), compounding maintenance risk.

By contrast, `Account.interestEarned` has the highest raw complexities (Cyclomatic 6, Cognitive 7) but low fan-in/out (1/1), so its change impact is more contained.

Q2.8: Your colleague Alice suggests the team to ignore refactoring of the code and focus the entire next Sprint in developing new features. Those new features will depend on the Bank class. Do you agree with Alice? Justify in terms of technical debt. Would your decision be the same if the suggestion was to refactor the Customer class?
It depends on the team's priorities. I would agree to postpone refactoring the `Bank` class if the sprint is time-constrained, since its complexity and fan-in values are relatively low, posing limited immediate risk.
However, the `Customer` class shows significantly higher complexity and coupling—especially the statementForAccount method—which increases maintenance effort and technical debt. Therefore, refactoring the Customer class should take priority to prevent future issues.

Q2.9: i) How is the readability of the project? You must ii) justify by highlighting functions or classes that have high / low readability; and iii) suggest how to improve the readability of the listed methods. (**Note:** You can assess this qualitatively by manual inspection of the code)

i) Overall readability:

The overall readability of the project is moderate but inconsistent across classes. Some methods are easy to follow, while others contain code smells and unclear naming that reduce understandability.

ii) Justification (high/low readability):

High readability: interestEarned() – the control flow is clear and logically structured, but it contains magic numbers (e.g., interest rate thresholds) that obscure meaning.

Low readability: statementForAccount() – suffers from poor naming and performs multiple responsibilities (formatting and computation), making it harder to read and maintain.

iii) Suggestions for improvement:

Replace magic numbers with named constants to make business logic self-explanatory.

Use descriptive method and variable names that clearly reflect their purpose.

Extract smaller helper methods to isolate distinct tasks (e.g., formatting output, computing totals).

Maintain consistent code formatting and comments for clarity.